

Documentação da rota GET

Visão geral

Esta rota retorna um pacote completo de dados administrativos vinculados à conta do usuário autenticado.

Ela foi desenhada para abastecer uma área de relatórios, painel administrativo ou exportação consolidada de informações da conta.

Método

GET

Objetivo

Buscar, em uma única requisição, os principais dados relacionados à conta do administrador autenticado:

- Conta
- Configurações da conta
- Clientes
- Vendedores
- Ordens de serviço
- Receitas
- Armações
- Lentes
- Anexos
- Histórico de status das ordens de serviço

Regras de acesso

A rota só pode ser acessada por usuários que atendam a todos estes critérios:

1. Estar autenticado
2. Ter perfil cadastrado na tabela `usuarios`
3. Estar com `status = "ativo"`
4. Ter `role = "admin"`

Qualquer falha nessas validações impede o retorno dos dados.

Fluxo da rota

1. Autenticação do usuário

A rota obtém o usuário autenticado via Supabase:

```
supabase.auth.getUser()
```

Possível erro

Se não houver usuário autenticado ou ocorrer erro na autenticação:

```
{  
  "error": "Usuário não autenticado."  
}
```

Status HTTP:

401 Unauthorized

2. Busca do perfil do usuário

Após autenticar, a rota consulta a tabela:

usuarios

Filtrando por:

```
id = user.id
```

Possível erro

Se o perfil não existir:

```
{  
  "error": "Perfil de usuário não encontrado."  
}
```

Status HTTP:

404 Not Found

3. Validação do status do usuário

A rota verifica se:

```
usuario.status === "ativo"
```

Possível erro

Caso o usuário esteja inativo ou bloqueado:

```
{  
  "error": "Seu acesso está inativo ou bloqueado."  
}
```

Status HTTP:

403 Forbidden

4. Validação da permissão de administrador

A rota permite acesso apenas se:

```
usuario.role === "admin"
```

Possível erro

Caso o usuário não seja administrador:

```
{  
  "error": "Acesso permitido apenas para administradores."  
}
```

Status HTTP:

403 Forbidden

5. Identificação da conta

A conta vinculada ao administrador é extraída de:

```
usuario.conta_id
```

Esse identificador é usado para buscar todos os dados relacionados à mesma conta.

6. Consulta paralela dos dados

A rota executa múltiplas consultas simultaneamente com:

```
Promise.all()
```

Isso melhora o desempenho em comparação com chamadas sequenciais.

Tabelas consultadas

Tabela	Filtro
<code>contas</code>	<code>id = contaId</code>
<code>configuracoes</code> <code>_conta</code>	<code>conta_id =</code> <code>contaId</code>
<code>clientes</code>	<code>conta_id =</code> <code>contaId</code>
<code>vendedores</code>	<code>conta_id =</code> <code>contaId</code>
<code>ordens_servico</code>	<code>conta_id =</code> <code>contaId</code>
<code>receitas</code>	<code>conta_id =</code> <code>contaId</code>
<code>armacoes</code>	<code>conta_id =</code> <code>contaId</code>

```
lentes          conta_id =  
                contaId
```

```
anexos          conta_id =  
                contaId
```

```
historico_sta   conta_id =  
tus_os          contaId
```

Resposta de sucesso

Status

200 OK

Corpo da resposta

```
{  
  "success": true,  
  "data": {  
    "conta": {},  
    "configuracoes_conta": {},  
    "clientes": [],  
    "vendedores": [],  
    "ordens_servico": [],  
    "receitas": [],  
    "armacoes": [],  
    "lentes": [],  
    "anexos": [],
```

```
"historico_status_os": []  
  
}  
  
}
```

Estrutura dos campos

Campo	Tipo	Descrição
success	boolean	Indica que a requisição foi concluída com sucesso
data.conta	object null	Dados da conta vinculada ao administrador
data.configuracoes_conta	object null	Configurações opcionais da conta
data.clientes	array	Lista de clientes da conta
data.vendedores	array	Lista de vendedores da conta
data.ordens_servico	array	Lista de ordens de serviço
data.receitas	array	Lista de receitas
data.armacoes	array	Lista de armações

<code>data.lentes</code>	<code>array</code>	Lista de lentes
<code>data.anexos</code>	<code>array</code>	Lista de anexos
<code>data.historico_status_os</code>	<code>array</code>	Histórico de status das ordens de serviço

Tratamento de erros internos

Se qualquer consulta ao banco falhar, a rota lança o erro e cai no bloco `catch`.

Resposta

```
{
  "error": "Erro interno ao carregar os dados dos relatórios."
}
```

Status

500 Internal Server Error

Códigos de resposta

St	Situação
200	
400	
401	
403	
404	
500	

20	Dados retornados com sucesso
40	Usuário não autenticado
40	Usuário inativo, bloqueado ou sem permissão de admin
40	Perfil de usuário não encontrado
50	Erro interno ao consultar os dados

Observações técnicas

A rota está bem montada, mas há dois pontos que eu ajustaria sem pensar duas vezes:

1. Uso de `select("*")`

Funciona, mas é um buffet livre sem limite. Em painéis administrativos, isso pode trazer dados demais, aumentar tempo de resposta e até expor colunas desnecessárias.

Melhor abordagem:

```
.select("id, nome, email, status")
```

quando possível.

2. Volume potencialmente muito alto

Essa rota retorna listas inteiras de clientes, ordens, anexos e histórico. Se a conta crescer, a resposta pode virar um caminhão tombado de JSON.

Para relatórios leves, tudo bem. Para produção robusta, vale considerar:

- Paginação
 - Filtros por período
 - Endpoint específico para agregações
 - Exportações separadas
-

Exemplo de uso no frontend

```
const response = await fetch("/api/admin/relatorios");
```

```
const result = await response.json();
```

```
if (!response.ok) {  
  throw new Error(result.error);  
}
```

```
console.log(result.data);
```

-